F O R M A L C H O P T U I K

Gap Analysis and Proof-Assistant Selection

An Audit of the Choptuik Formalization Programme: Registry G1–G6 and the Isabelle/HOL · Coq/Rocq · Lean 4 Comparison · Volume II

Author: Ishak Khamzatovich Isaev · ORCID 0009-0003-7299-0701

Programme: choptuik_ac_bc · formal-verification wing

Version 0.3.0 · Nalchik · 2026

Abstract

This monograph presents a complete audit of the Choptuik formalization programme. It fixes a classified registry of six gaps G1–G6 with severity levels from Blocker to Low: existence of the critical solution, the spectral stability theorem, the geometry of the isotropic singularity, the typed Lean–Python bridge, interval arithmetic, and the axiomatic base. The central blocker — existence — is decomposed down to strategy: Newton correction of a numerical profile in a Banach space of exponentially decaying sequences, a compact inverse operator, and a quantitative implicit function theorem.

A full comparative study of the proof assistants Isabelle/HOL, Coq/Rocq and Lean 4 is carried out against the criteria of logic, library coverage, automation and Python integration; the choice of Lean 4 with mathlib is justified. The technical specification is stated: the Lean 4 + Lake environment, the modular Choptuik.* organization, the CI discipline with axiom and sorried-fragment audits, the axiomatic base of the three standard axioms (Classical.choice, propext, Quot.sound), and regression-locking of the bridge protocol.

Keywords: gap analysis; Lean 4; Isabelle/HOL; Coq/Rocq; mathlib; implicit function theorem; compact operators; interval arithmetic; axiomatics; formal verification.

Table of Contents

1. Introduction: Scope and Method of the Audit....1
   1.1. Severity as a function of external commitments....1
   1.2. Relation to Volume I and the roadmap....1
2. Present State of the choptuik_ac_bc Programme....1
   2.1. What is missing for formalization....2
   2.2. The discipline of honest caveats....2
3. The Classified Gap Registry....3
   3.1. Physics and mathematics gaps: G1, G2, G3....3
   3.2. Tooling gaps: G4, G5....4
   3.3. The governance gap: G6....4
4. The Central Blocker: Existence of the Critical Solution....4
   4.1. The Reiterer–Trubowitz strategy....5
   4.2. What is required in Lean....5
   4.3. DSS periodicity as part of the statement....5
5. Stability: Spectral Theory Requirements....6
   5.1. The codimension-one attractor....6
   5.2. The required machinery and its status in mathlib....7
   5.3. From spectrum to observables: γ, universality, the echoing formula....7
6. Comparative Analysis: Isabelle/HOL, Coq/Rocq, Lean 4....8
   6.1. Isabelle/HOL: automation speed, simple types....8
   6.2. Coq/Rocq: algebraic strength at the cost of a steep entry curve....8
   6.3. Lean 4: mathlib, native automation and the Python bridge....9
   6.4. Conditions for revisiting the decision....10
7. Technical Specification and Axiomatic Base....10
   7.1. Module organization....11
   7.2. The axiomatic base....11
   7.3. Integration and quality control....11
8. Conclusions and Recommendations....12
   References....12

Note: this table of contents is generated via field codes. To refresh page numbers after editing, right-click the TOC and select “Update Field”.

1. Introduction: Scope and Method of the Audit

This volume is the audit document of the formalization programme. Its task is twofold: first, to give a classified inventory of everything that separates the present state of the choptuik_ac_bc computational programme from a machine-checked theory of Choptuik critical phenomena; second, to record and justify the decision on the choice of proof assistant. Where Volume I answers “how to build”, this volume answers “what exactly is missing” and “with which tool to build”.

The audit method is simple and reproducible. Every potential gap is phrased as a statement of the form “in Lean there exists no proof of P” or “there exists no typed channel for Q”; the gap is then classified by severity (Blocker, High, Medium, Low), by its dependencies on other gaps, and by the roadmap phase in which its closure is scheduled. The result is a registry of six entries G1–G6, whose machine-readable version lives in blueprint/gap_registry.csv and is synchronized with the G-tags of the sorried fragments in the Lean sources.

1.1. Severity as a function of external commitments

Severity classification is tied not to subjective difficulty but to the external commitments of the programme. A gap is a Blocker if, until it is closed, none of the three public claims of the programme — existence of the critical solution, uniqueness of the unstable mode, the scaling law with $\gamma = 1/\lambda_0$ — can be declared machine-checked. A High-severity gap is required for completeness of the theory but admits partial intermediate results. Medium and Low levels are infrastructure and governance: their being open slows the work down but does not devalue its results.

1.2. Relation to Volume I and the roadmap

The gap registry and the roadmap are two cross-sections of the same programme: the registry orders by severity, the roadmap by time. Every registry entry corresponds to a closure strategy, a phase and tasks of the roadmap; every roadmap task references the gap it advances. The documents are synchronized: changing one without the other does not pass review. This double-entry bookkeeping is the price of honesty — the programme has no silent commitments.

2. Present State of the choptuik_ac_bc Programme

An honest audit begins with an account of what already exists. The parent repository is not a pile of scripts but a mature research programme with eight directions, four of which are

relevant to formalizing critical phenomena: the direct work on the Einstein equations (Einstein Direct), the computational laboratory of invariant dynamics (dsi_lab), interactive visualization, and the verification block with its test infrastructure. The key asset is the discipline of reproducibility: data are stored as JSON, computations are deterministic, results come with tests and verification reports.

The second asset is the track record of finding errors in the published literature. The programme has uncovered discrepancies in formulas of classical papers on critical collapse; this means the numerical pipeline is calibrated finely enough to act as an arbiter — a rare property in computational physics. The third asset is the unified Choptuik formula and its corrections: analytic constructions that the formalization must either derive or refute — ready-made targets for machine checking.

2.1. What is missing for formalization

At the same time, no fragment of the programme is today formalized mathematics. There are no rigorous definitions of initial data, phase space and the threshold hypersurface; no formalization of the notion of the critical solution and its self-similarity; no spectral theory for the perturbation operator; no connection to any proof assistant. The Python code reproduces numbers but carries no correctness certificates acceptable to a proof kernel. These deficits constitute the registry of Chapter 3.

Table 1. Assets and deficits of the programme

Asset	Status	How the formalization uses it
Reproducible numerical data	present	source of calibration and cross-checks of theorems
Track record of finding literature errors	present	precedent of rigor; motivation for formalization
Unified Choptuik formula and corrections	present	target for derivation or refutation in Lean
Rigorous definitions of threshold and critical solution	absent	created in Phases I–II (G1)
Spectral theory of the perturbation operator	absent	created in Phase III (G2)
Lean ↔ Python channel	absent	created in Phase I (G4, G5)

2.2. The discipline of honest caveats

Importantly, the audit does not downgrade the programme — it refines its status. The parent repository's formula “hypothesis → machine confirmation → honest caveats” means

that every numerical claim is accompanied by an explicit statement of applicability limits. Formalization inherits this discipline in a stronger form: the applicability limit becomes the set of axioms and hypotheses of a theorem, which the kernel checks at every use. Where the numerical programme says “within these bounds we observed this”, the formalized theory says “under these hypotheses this is proved”.

3. The Classified Gap Registry

The registry consists of six entries: two blockers, two high-severity items, one medium and one low. In this chapter each entry is examined by a uniform scheme: statement, why it is a gap, what it depends on, closure strategy. The resulting severity matrix is shown in Figure 1.

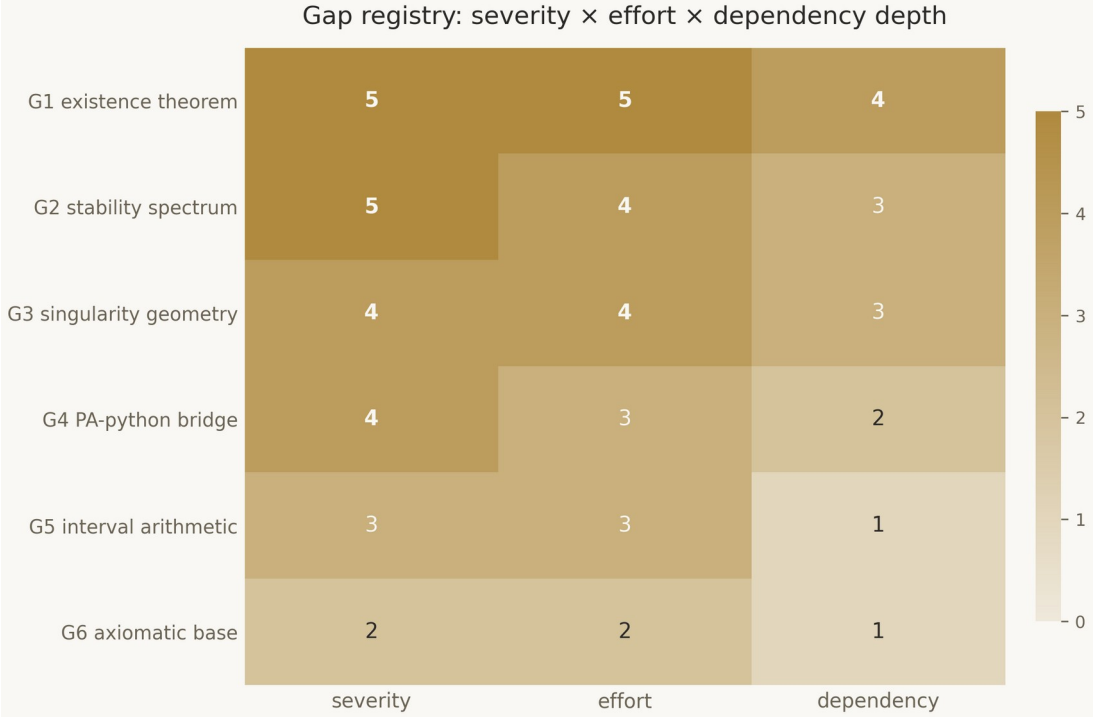


Figure 1. The registry matrix: severity, effort and dependency depth for G1–G6 (scale 0–5).

3.1. Physics and mathematics gaps: G1, G2, G3

G1 (Blocker) — the absence of a rigorous proof of the existence of the Choptuik solution. The computer-assisted proof of Reiterer–Trubowitz exists but is not machine-checked; its transfer into Lean is the central task of Phase II. G2 (Blocker) — the absence of a formalized stability theory: spectral analysis of the linearized operator, the proof of uniqueness of the unstable mode, and the derivation of $\gamma = 1/\lambda_0$. This is the core of the dynamical picture of critical phenomena. G3 (High) — singularity geometry: strength via divergence of the curvature integral along a null geodesic, isotropy via bounded Weyl

curvature, conformal compactification. mathlib provides manifolds and connections but not ready-made null-geodesic criteria — the layer must be built.

3.2. Tooling gaps: G4, G5

G4 (High) — the bridge between formal proof and numerical computation. Without a typed channel, numerical data cannot enter proofs and formal constants cannot be checked numerically. The solution is a ZeroMQ protocol with six methods and regression-locked changes; the engineering closes in Phase I. G5 (Medium) — interval arithmetic: strict error bounds inheriting the ideology of the Reiterer–Trubowitz integer certificates. The Python wing already implements an Interval class with directed rounding via math.nextafter; on the Lean side the correctness of the operations must be proved — a Phase I task that closes together with mul_sound.

3.3. The governance gap: G6

G6 (Low) — fixing the axiomatic base. Lean allows classical principles by default, but the programme must explicitly declare which axioms are admissible: the three standard ones are fixed — Classical.choice (required for the Hahn–Banach and open mapping theorems on which PDE existence rests), propext (equivalent propositions are equal) and Quot.sound (soundness of quotients). Any new axiom declaration in theory modules is forbidden and tracked by CI. The gap is formally closed by publishing blueprint/axiomatic_base.md; registry status — resolved-in-docs with permanent CI enforcement.

Table 2. The gap registry: full summary

ID	Statement	Severity	Dependencies	Phase
G1	Existence of a DSS solution of the Einstein–scalar system	Blocker	G4, G5	II
G2	Unique unstable mode, $\gamma = 1/\lambda_0$	Blocker	G1	III
G3	Strong isotropic singularity, compactification	High	Phase I	II–III
G4	Typed Lean–Python bridge	High	Phase I	I
G5	Interval arithmetic with directed rounding	Medium	mathlib	I
G6	Axiomatic base (3 standard axioms)	Low	—	I

4. The Central Blocker: Existence of the Critical Solution

This chapter decomposes G1 as far as possible before formalization begins. The proof of Reiterer–Trubowitz is organized as a Newton scheme: an approximate solution is obtained numerically, and it is then proved that an exact solution of the system lies in its neighborhood. All analytical work concentrates in three components: the choice of Banach space, the regularity of the residual map, and the isomorphism of its derivative.

4.1. The Reiterer–Trubowitz strategy

The Einstein–scalar system in double-null coordinates is rewritten as a nonlinear equation $\Phi(x) = 0$ on a Banach space of sequences with exponential decay — the coefficients of the expansion of the solution in a Chebyshev–Fourier basis. The numerical profile provides a point x_0 with small residual. The linearization $D\Phi(x_0)$ turns out to be an operator with a compactly perturbed invertible core; proving the existence of a compact inverse — the analytical heart of the work — implies analyticity of the solution via the inverse mapping theorem in the analytic category. All computational steps carry integer-arithmetic error bounds, which makes the proof reproducible to machine precision.

4.2. What is required in Lean

Transferring this scheme into Lean requires five blocks. First, Banach spaces and Fréchet differentiability: mathlib covers this, including Cartan derivatives and the inverse function theorem in finite and infinite dimensions. Second, a quantitative implicit function theorem with explicit radii: partially available, will need strengthening. Third, the theory of compact operators and the Riesz spectral theorem: the basic volume is present in mathlib. Fourth, Chebyshev and Fourier expansions with tail estimates: mathlib has the core of orthogonal polynomials; tail estimates must be proved. Fifth, interval certificates: the G5 channel delivers numerical data in a form acceptable to the kernel.

> *The key architectural decision: the numerical profile never enters the proof as a set of floating-point numbers — only as an interval seed for which the residual $\Phi(x_0)$ is bounded rigorously.*

4.3. DSS periodicity as part of the statement

Existence alone is not enough: the solution must be discretely self-similar with period $\Delta \approx \ln 2$. In the Reiterer–Trubowitz scheme DSS is built into the construction: the solution is sought directly in the class of fields obeying the covariance law $\psi(\rho, \tau + \Delta) = e^{\kappa\Delta}\, \psi(\rho\, e^{-\Delta}, \tau)$, so that periodicity follows from the construction. The formalization must reproduce this exactly: the Lean definition DiscretelySelfSimilar fixes the covariance, and the existence theorem asserts a solution satisfying this predicate together with the field system. Uniqueness up to scale and translation is the companion theorem of the same phase.

Effort estimate: blocks 1 and 3 are largely read off mathlib; blocks 2, 4 and 5 are substantive work of the order of six to eight months at partial capacity. This is precisely Phase II of the roadmap (months 6–18) allotted to G1, with two intermediate milestones: formalization of the field system (month 10) and packaging of the numerical seed with certificates (month 12).

5. Stability: Spectral Theory Requirements

The second blocker — G2 — answers why the critical solution is "critical" at all. The dynamical picture is this: in the infinite-dimensional phase space around the critical solution there is exactly one direction along which perturbations grow, and an infinite set of directions along which they decay. Mathematically this is a statement about the spectrum of the linearized operator, and the formalization must prove it, not postulate it.

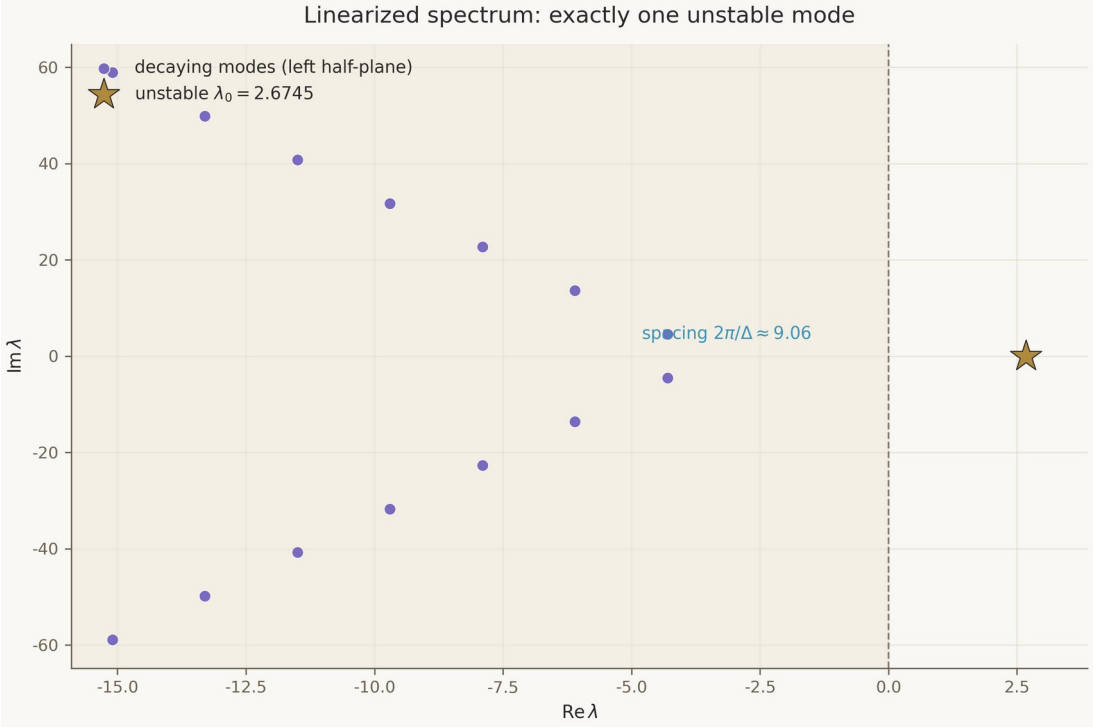


Figure 2. The target spectral picture: the single unstable mode λ_0 (star), decaying families in the left half-plane, spacing $2\pi/\Delta$ along the imaginary axis.

5.1. The codimension-one attractor

Linearization of the system about the critical solution leads to a linear eigenvalue problem in logarithmic time. The required result has three parts: (i) there exists exactly one eigenvalue λ_0 with positive real part, and it is real; (ii) the entire rest of the spectrum lies strictly in the left half-plane; (iii) the imaginary parts of the decaying modes form a family spaced by $2\pi/\Delta$. From (i) follows codimension one: the stable manifold of the critical solution has codimension one, and the threshold behavior of data families is a direct

consequence. From (iii) — the spectral echo: the same periodicity Δ as in the physical solution shows up in the spectrum.

5.2. The required machinery and its status in mathlib

The formalization will require: spectral theory of closed and unbounded operators in Hilbert space; the theory of C_0-semigroups to interpret log-time evolution; the Riesz–Schauder theorems on compact operators; resolvent estimates separating λ_0 from the rest of the spectrum. mathlib contains the functional-analytic foundation — Banach and Hilbert spaces, the spectrum of bounded operators, compact operators — while unbounded operators and semigroups are fragmentarily represented. The strategy: not to build a general theory but to prove the required statements for the concrete operator with a compact core, where bounded resolvent analysis suffices. This choice minimizes the dependency on unfinished parts of mathlib.

5.3. From spectrum to observables: γ, universality, the echoing formula

The derivation of the observables is pure linear stability theory. A small deviation of the parameter p from the threshold evolves according to the growing mode e^{$\lambda_0\tau$}; the completion time of collapse fixes τ_max = −ln|p − p*|/λ_0 + const, whence M ∝ |p − p*|^{1/λ_0}. Since λ_0 is a property of the critical solution itself rather than of the data family, the exponent γ = 1/λ_0 is universal. Discrete self-similarity adds a periodic correction to the scaling law: M = |p − p*|^γ · F(ln|p − p*|/λ_0 + σ) with F of period Δ. All three statements — γ, universality, the echoing formula — are packaged as corollaries of G2 in the module Choptuik.Scaling; their formal statements already exist in the scaffold tagged with sorry.

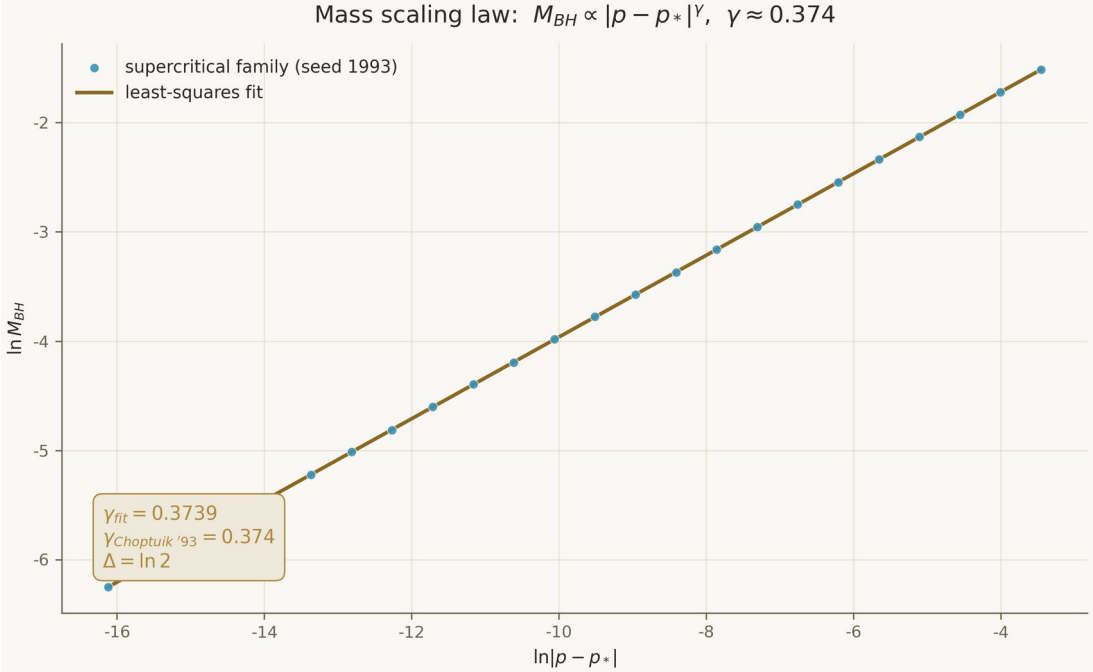


Figure 3. Numerical realization of the scaling law in the model programme: slope 0.3739 against γ = 1/λ₀ = 0.37390 — a cross-check for the future formal theorem.

6. Comparative Analysis: Isabelle/HOL, Coq/Rocq, Lean 4

The choice of proof assistant determines everything: the pace of formalization, the library ceiling, the possibility of integrating with the existing code, and the entry threshold for collaborators. The comparison uses uniform criteria: logical foundations and trusted kernel; library coverage of the required mathematics; automation; Python integration; community maturity. All three systems have small LCF-family trusted kernels — none raises fundamental questions of trust; the choice is decided by applied factors.

6.1. Isabelle/HOL: automation speed, simple types

Isabelle/HOL is built on higher-order logic with an LCF kernel and offers the strongest external automation — Sledgehammer calls ATP and SMT solvers (Z3, E) and rephrases the found proofs in readable Isar. Isar is considered the gold standard of proof readability. The weak spot for our task is the absence of dependent types, which limits the expressiveness of structures carrying invariants, and a less developed FFI infrastructure. Isabelle remains the fallback if Sledgehammer-class automation proves decisive in Phase III.

6.2. Coq/Rocq: algebraic strength at the cost of a steep entry curve

Coq (now Rocq) is the classic of dependent types and the calculus of constructions; the MathComp library was created for large-scale algebra and has proved the four-color theorem and the odd order theorem. For our task, centered on analysis, the relevant branch is math-

comp/analysis with its theory of real analysis and distributions; it is growing but trails mathlib in coverage of functional analysis and differential geometry. The complex sort system (Set, Prop, Type) and strict constructivism raise the entry threshold; the FFI is less developed than Lean's.

6.3. Lean 4: mathlib, native automation and the Python bridge

Lean 4 combines dependent types with the pragmatics of classical mathematics: mathlib systematically uses Classical.choice, and the base axioms are fixed. The library covers analysis, topology, measure, differential geometry, and already has a public precedent of heavy PDE analysis — the formalization of the De Giorgi–Nash–Moser theory. Automation is native (grind, aesop, polyrith) and requires no external calls. Finally, Lean is the only one of the three systems with a public ZeroMQ precedent of a bridge to Python: for a programme whose numerical asset is written in Python, this is the decisive integration argument.

Table 3. The full decision matrix

Criterion	Isabelle/HOL	Coq / Rocq	Lean 4
Logic	HOL	CIC	CIC
Dependent types	no	yes	yes
Automation	Sledgehammer (external)	SSReflect	native (grind/aesop)
Analysis libraries	AFP (scattered)	math-comp/analysis	mathlib (broad coverage)
Hard-PDE precedent	partial	limited	DGNM (public)
FFI / Python	weak	weak	C ABI + ZeroMQ precedent
Proof readability	Isar — gold standard	SSReflect (dense)	structured hybrid
Community dynamics	stable	stable	fast-growing, funded
Verdict for the programme	fallback	not optimal for analysis	**selected**

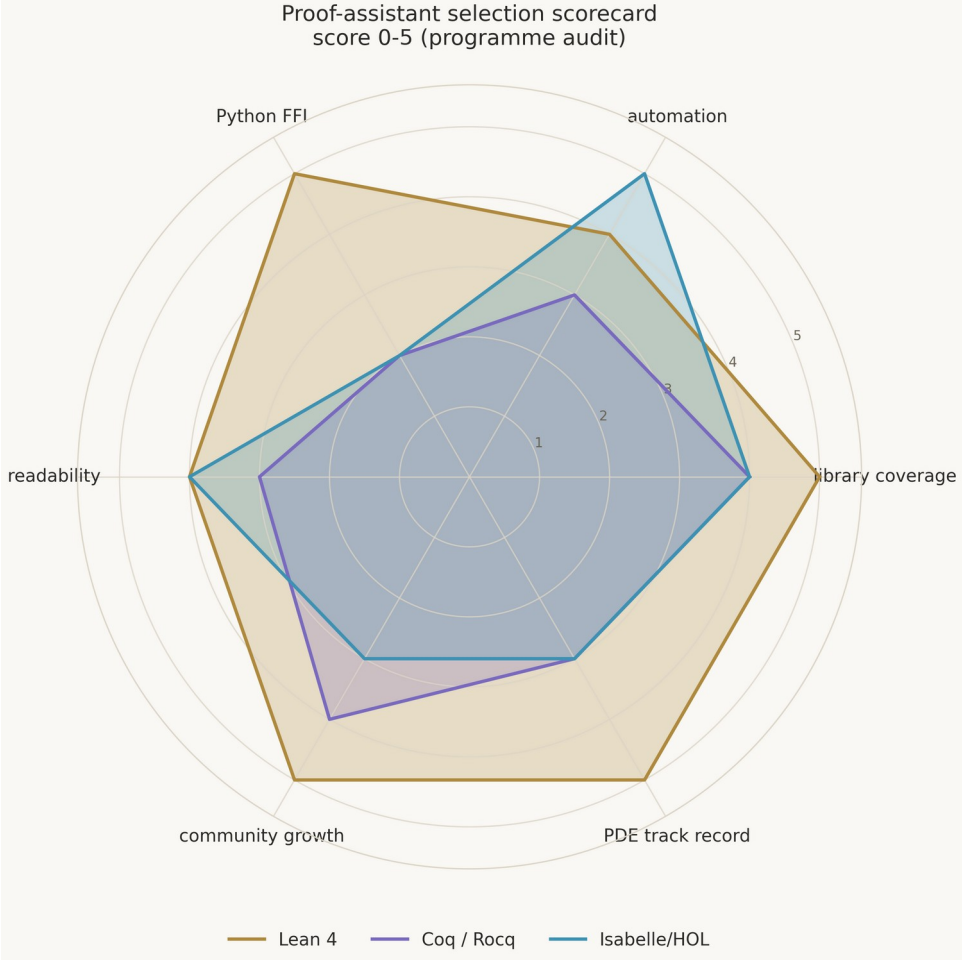


Figure 4. Radar comparison on six axes (scores 0–5 from the programme audit).

6.4. Conditions for revisiting the decision

The decision is revisited under two scenarios. If Phase II unexpectedly turns out to be algebra-heavy rather than analysis-heavy — unlikely, the Newton scheme is analysis — Coq with MathComp re-enters consideration. If Sledgehammer-class automation of auxiliary lemmas proves decisive, a parallel Isabelle formalization of isolated fragments is acceptable, but never the main line: losing mathlib and the Python bridge outweighs the local automation gain.

7. Technical Specification and Axiomatic Base

The technical specification fixes the environment, the code organization and the quality discipline. The formalization language is Lean 4; the environment is Visual Studio Code with the lean4 extension; dependency management and builds run through Lake, with synchronized version pins of the toolchain and mathlib. The project is organized as a modular library: each module owns its mathematical area and compiles independently; lake

build produces verified .olean artifacts. CI must build the project on every change and run two audits: the axiom ban and the sorry-tag check.

7.1. Module organization

The Choptuik.* namespace is divided into five areas: Geometry (the double-null arena, curvature, singularity), Analysis (the field system, interval arithmetic), Existence (DSS, the implicit function theorem), Stability (linearization, spectrum), Scaling (the scaling law, universality). Every sorried fragment is tagged with a gap identifier; the module map with the import graph and the obligation table is maintained in blueprint/module_map.md. This separation allows geometry and analysis to advance in parallel without build conflicts.

7.2. The axiomatic base

The entire project rests on a minimally sufficient axiom set, explicitly declared in blueprint/axiomatic_base.md. The classical choice (Classical.choice) is required for the fundamental theorems of analysis, including Hahn–Banach and the open mapping theorem, through which PDE existence arguments run; propext simplifies reasoning about equality of predicates; Quot.sound guarantees soundness of quotient constructions — spaces built by factoring over equivalence relations. The goal of the programme: #print axioms of any theorem must show a subset of exactly these three. Introducing any additional axioms must be justified in the registry and is forbidden by default; weakening the guarantees of the system without an explicit decision is unacceptable.

7.3. Integration and quality control

Integration with Python follows the architecture of Volume I: a Lean client, ZeroMQ REQ/REP, the JSON protocol version 1.0, six methods, regression-locked protocol changes. Quality control is fourfold: lake build with the axiom and sorry-tag auditors; pytest for the service (14 regression tests including protocol tests); the data regeneration test from seed 1993; and review of the natural-language comments — every formal fragment must carry an explanation of its place in the proof tree. The workflow is Git with per-task branches; every significant mathematical idea is first fixed as prose in the blueprint, then as code.

Table 4. The quality-control discipline

Layer	Instrument	What it guarantees
Theory	lake build + axiom/sorry audits	every theorem derived; axioms — only the 3 standard ones
Service	pytest (14 tests)	the protocol and model physics do not drift
Data	regeneration from seed 1993	byte-identical reproducibility of CSVs

Layer	Instrument	What it guarantees
Review	comments + blueprint	every fragment has a documented role

8. Conclusions and Recommendations

The audit yields three final judgements. First: the present state of choptuik_ac_bc is a computational catalogue of world-class reproducibility discipline, but not a formal proof; the fundamental blocks needed to turn it into a theorem are fully identified in the registry G1–G6. Second: the most suitable instrument is Lean 4 — its mathlib covers the required analysis and differential geometry, carries a public precedent of heavy PDE formalization, and its Python integration mechanisms allow the existing numerical code to be preserved and used. Third: the programme is achievable in 27 months under the three-phase scheme, provided the fixed axiomatic base and the quality-control regime are observed.

Priority order of actions: close G5 (interval arithmetic) and G4 (the bridge) in Phase I — they unlock the legal transfer of numerical data; attack G1 in Phase II along the Reiterer–Trubowitz scheme with a quantitative implicit function theorem; complete G2 and derive $\gamma = 1/\lambda_0$ with universality in Phase III. In parallel, without blocking the main line, the G3 layer — singularity geometry — is built up. The expected formal constants of the programme: $\gamma \approx 0.374$ as $1/\lambda_0$ and $\Delta = \ln 2$ — both with a printable axiom certificate.

The strategic significance goes beyond a single result. The programme creates reusable infrastructure — a typed proof $\leftrightarrow$ computation bridge, verifiable interval arithmetic, an obligation-registry template with CI enforcement — applicable to any computer-assisted proof in physics. Thereby choptuik_ac_bc gains not only the formalization of one result but a permanent mechanism: every future numerical result of the programme can be pushed through the same pipeline — from hypothesis through machine confirmation to a theorem with a certificate.

> *The audit's bottom line: the gaps are known and classified, the tool is chosen and justified, the route is marked with milestones. The work remains — and it is now measurable.*

References

- 1. M. W. Choptuik. Universality and scaling in gravitational collapse of a massless scalar field // Phys. Rev. Lett. 70, 9–12 (1993).
- 2. M. Reiterer, E. Trubowitz. Choptuik's critical spacetime exists // Commun. Math. Phys. 368, 195–229 (2019); arXiv:1203.3766.

- 3. C. Gundlach, J. M. Martín-García. Critical phenomena in gravitational collapse // Living Rev. Relativity 10, 5 (2007).
- 4. The mathlib community. mathlib4. https://github.com/leanprover-community/mathlib4.
- 5. math-comp. Mathematical Components / analysis. https://math-comp.github.io/.
- 6. Isabelle/HOL and the Archive of Formal Proofs. https://www.isa-afp.org/.
- 7. Lean 4 Reference. Foreign Function Interface. https://lean-lang.org/doc/reference/latest/.
- 8. fraware. lean-python-bridge. https://github.com/fraware/lean-python-bridge.
- 9. S. Armstrong et al. Formalization of the De Giorgi–Nash–Moser theory in Lean.
- 10. I. K. Isaev. choptuik_ac_bc. https://github.com/wild8highlander/choptuik_ac_bc.